SIMATS ENGINEERING

ASSESSMENT TOOL 1

Coding Challenge

COURSE CODE: ITA0402 COURSE NAME: Statistics with R Programming

COURSE OUTCOME COVERED

CO2: R Programming Structures, Different types of loops such as: for (), while() loops, Control Statements, Looping Over Non-Vector Sets, - If-Else, Arithmetic and Boolean Operators and values, Default Values for Argument, Return Values, Deciding Whether to explicitly call return- Returning Complex Objects, Functions are Objective, No Pointers in R, Recursion. (BL4-BL5)

Assessment Tool Weightage: 25%

QUESTIONS: 10 **Total Marks: 50**

Coding Challenge

Code of Conduct:

*I **RAKSHITH R.R (192324156)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Coding Challenge

1. Write an R program to declare variables of each basic data type (numeric, integer, character, logical, complex) and print their class using class().
2. Create a list containing a vector, a matrix, and a character string. Write a program to access and print each element of the list individually.
3. Write a program to create a 4x4 matrix of integers and perform matrix addition, subtraction, and multiplication with another 4x4 matrix.
4. Create a 3-dimensional array of size 2x3x4 filled with random numbers, then write a program to extract and print a specific slice of the array.
5. Define an S3 class called Student with fields name, age, and marks, and write a method to display student details. Then implement the same using an S4 class and compare the outputs.
6. Write an R program using a for loop to print the Fibonacci series up to n terms, then rewrite the same using a while loop.
7. Write a program using nested if-else statements to categorize a student's grade based on marks entered by the user.
8. Write a recursive R function to calculate the factorial of a number, and another recursive function to find the GCD of two numbers.
9. Write a function in R with default argument values that calculates the simple interest, and demonstrate calling it with and without passing all arguments.
10. Write a program to loop over a non-vector set (like a list of mixed data types) and print each element along with its data type.
11. Create a data frame with columns StudentID, Name, Marks, and Grade for 10 students. Write a program to add a new column Result based on a condition on Marks.
12. Write a program to merge two data frames — one containing student names and IDs, and another containing student IDs and marks — using a common key column.
13. Write a program to demonstrate melting and casting of a wide-format data frame into long format and back, using the reshape2 package.
14. Write an R program to read data from a CSV file into a data frame, clean missing values, and write the cleaned data frame back to a new CSV file.
19. Write a program to create an atomic vector of 10 numeric values and demonstrate at least 5 different vector operations (sum, mean, sorting, reversing, indexing).
20. Write a program to create two logical vectors and demonstrate AND, OR, NOT, and XOR operations on them element-wise.
21. Write a program to create a factor variable from a character vector representing blood groups, and display its levels and frequency table.

22. Write a program to create a named list representing an employee record (name, id, salary, department) and update one field's value.
23. Write a program to demonstrate the difference between a Reference Class and an S4 class by creating a BankAccount object with deposit() and withdraw() methods.
24. Write a program to create a character vector of names, then convert it to uppercase, lowercase, and count the number of characters in each name.
25. Write a program using a for loop to check whether each number in a given vector is prime or not.
26. Write a program to reverse a string using recursion (without using the built-in reverse function).
27. Write a program using while loop and break/next statements to print only even numbers between 1 and 50, skipping multiples of 10.
28. Write a recursive function to compute the sum of digits of a given number.
29. Write a program with a function that returns multiple values (as a list) — minimum, maximum, and average of a numeric vector — and demonstrate calling it.
30. Write a program to implement a simple calculator using a function with arguments for two numbers and an operator (+, -, *, /), using if-else control statements.
31. Write a recursive R function to compute the nth term of a geometric progression given the first term and common ratio.
32. Write a program to check if a given matrix is symmetric using nested loops and control statements.
33. Write a program to create a data frame of employee records and use apply()/sapply() functions to compute the total salary column-wise.
34. Write a program to sort a data frame of student records by marks in descending order and display the top 5 students.
35. Write a program to filter rows in a data frame based on multiple conditions (e.g., Marks > 60 AND Attendance > 75).
36. Write a program to read a dataset from a text file, split one column into two new columns (e.g., splitting a "Full Name" into First and Last Name), and save the result.
37. Write a program to remove duplicate rows from a data frame and count how many duplicates were removed.
38. Write a program to add, rename, and delete columns in an existing data frame of product sales data.
39. Write a program using a for loop combined with if-else to print Pascal's Triangle up to n rows.
40. Write a program to implement a recursive function that calculates the power of a number (base^exponent) without using the ^ operator.
41. Write a program using a while loop to implement a simple guessing game where the user guesses a number until they get it correct, tracking the number of attempts.

42. Write a function in R that explicitly uses return() to exit early from a loop when a specific condition is met (e.g., stop searching a vector once a target value is found).
43. Write a recursive function to generate all Fibonacci numbers less than a given limit n, and explain why recursion is less efficient than looping for this case.
44. Write a program demonstrating a function that returns a complex object (a list containing a vector, a string, and a boolean) and access its individual components after the call.
45. Write a program to create a data frame from three separate vectors (Name, Department, Salary) and convert it into a matrix, explaining any changes in data type.
46. Write a program to group a data frame by a categorical column (e.g., Department) and compute the average salary for each group using aggregate() or tapply().
47. Write a program to write a data frame to a .txt file with a custom delimiter (e.g., "|"), then read it back into R and verify the data matches.
48. Write a program to reshape a data frame from long format to wide format using dcast(), based on a dataset containing StudentID, Subject, and Marks.
49. Write a program to identify and count missing values (NA) column-wise in a data frame, then impute them using the column mean.
50. Write a program to join two data frames using merge() with an outer join (full join) and explain how missing matches are represented in the result.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Excellent)	Level 3 (Good)	Level 2 (Satisfactory)	Level 1 (Needs Improvement)
Data Preparation	15%	Dataset is correctly created/imported with no errors.	Dataset is mostly correct with minor errors.	Dataset contains some inaccuracies affecting calculations.	Dataset is incorrect or incomplete.
Calculation Accuracy	35%	All statistical measures/calculations are correct and accurately interpreted.	Most calculations are correct with minor errors.	Several calculation errors present.	Major errors; calculations are incorrect or missing.
Use of R Functions	20%	Appropriate R functions are selected and used efficiently.	Functions are mostly appropriate with minor issues.	Limited or partially correct use of functions.	Incorrect or no use of relevant R functions.
Analysis and Interpretation	20%	Results are clearly interpreted with logical conclusions.	Interpretation is mostly correct with minor gaps.	Limited interpretation of results.	No meaningful interpretation provided.
Presentation of Results	10%	Output is clearly organized, labeled, and documented.	Output is organized with minor formatting issues.	Output lacks clarity or proper labeling.	Output is poorly presented or incomplete.

Question 1

Write an R program to declare variables of each basic data type (numeric, integer, character, logical, complex) and print their class using class().

Approach:

One variable of each basic data type is declared (numeric, integer, character, logical, complex). They are stored in a named list and a for loop prints each value with its class() so the type of every variable is shown.

Code:

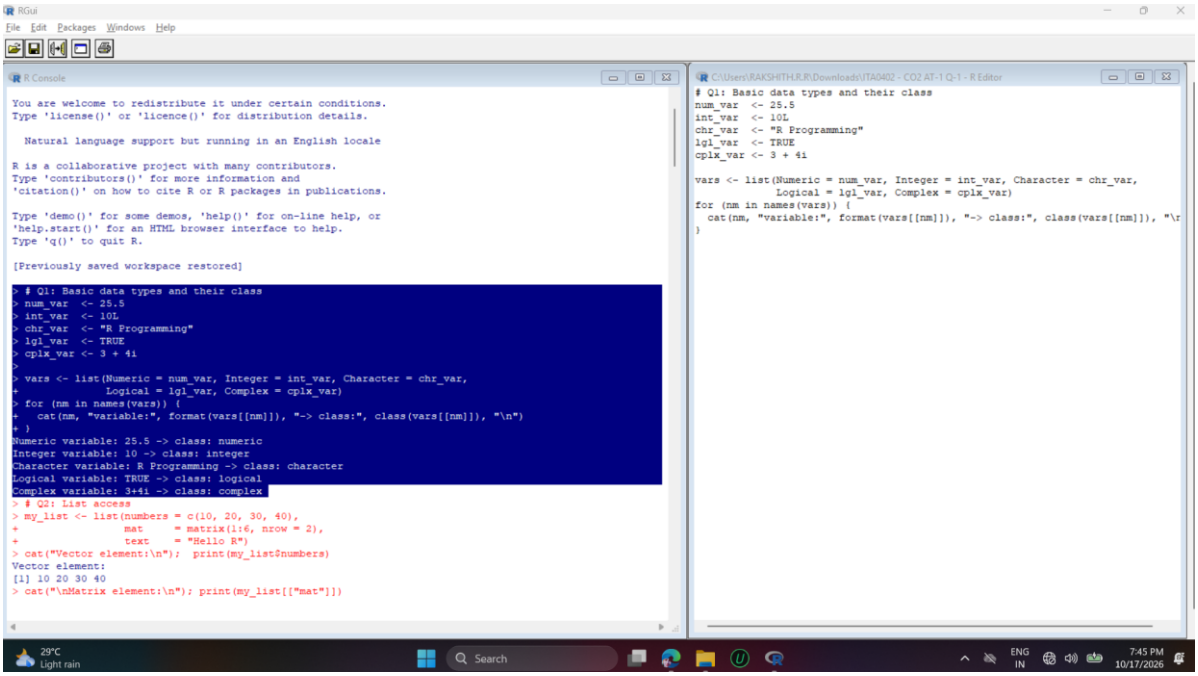
```
# Q1: Basic data types and their class
num_var  <- 25.5
int_var  <- 10L
chr_var  <- "R Programming"
lgl_var  <- TRUE
cplx_var <- 3 + 4i

vars <- list(Numeric = num_var, Integer = int_var, Character = chr_var,
             Logical = lgl_var, Complex = cplx_var)
for (nm in names(vars)) {
  cat(nm, "variable:", format(vars[[nm]]), "-> class:", class(vars[[nm]]), "\n")
}
```

Output:

```
Numeric variable: 25.5 -> class: numeric
Integer variable: 10 -> class: integer
Character variable: R Programming -> class: character
Logical variable: TRUE -> class: logical
Complex variable: 3+4i -> class: complex
```

Output Screenshot:



Interpretation:

class() confirms how R stores each value: 25.5 is numeric, 10L is integer (the L suffix forces integer), the text is character, TRUE is logical and 3+4i is complex.

Question 2

Create a list containing a vector, a matrix, and a character string. Write a program to access and print each element of the list individually.

Approach:

A list is created holding a numeric vector, a 2 x 3 matrix and a character string. Each element is accessed individually using the $ operator, the [["name"]] form and the [[index]] form.

Code:

```
# Q2: List access
my_list <- list(numbers = c(10, 20, 30, 40),
                mat     = matrix(1:6, nrow = 2),
                text    = "Hello R")
cat("Vector element:\n");  print(my_list$numbers)
cat("\nMatrix element:\n"); print(my_list[["mat"]])
cat("\nString element:\n"); print(my_list[[3]])
```

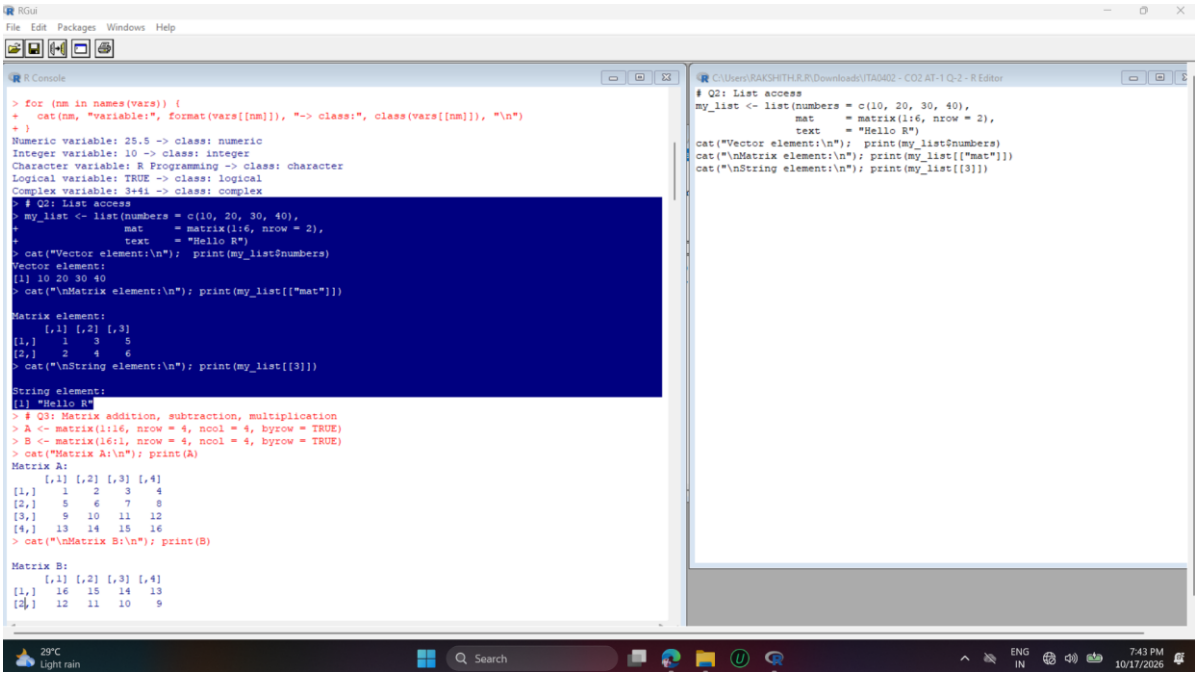
Output:

```
Vector element:
[1] 10 20 30 40

Matrix element:
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6

String element:
[1] "Hello R"
```

Output Screenshot:



Interpretation:

A list can hold objects of different types in one structure. The vector, matrix and string were each retrieved by name or by position without affecting the others.

Question 3

Write a program to create a 4x4 matrix of integers and perform matrix addition, subtraction, and multiplication with another 4x4 matrix.

Approach:

Two 4 x 4 integer matrices are created with matrix(). Addition and subtraction use the element-wise + and - operators, and matrix multiplication uses the %*% operator.

Code:

```
# Q3: Matrix addition, subtraction, multiplication
A <- matrix(1:16, nrow = 4, ncol = 4, byrow = TRUE)
B <- matrix(16:1, nrow = 4, ncol = 4, byrow = TRUE)
cat("Matrix A:\n"); print(A)
cat("\nMatrix B:\n"); print(B)
cat("\nA + B:\n"); print(A + B)
cat("\nA - B:\n"); print(A - B)
cat("\nA x B (matrix multiplication):\n"); print(A %*% B)
```

Output:

```
Matrix A:
     [,1] [,2] [,3] [,4]
[1,]    1    2    3    4
[2,]    5    6    7    8
[3,]    9   10   11   12
[4,]   13   14   15   16

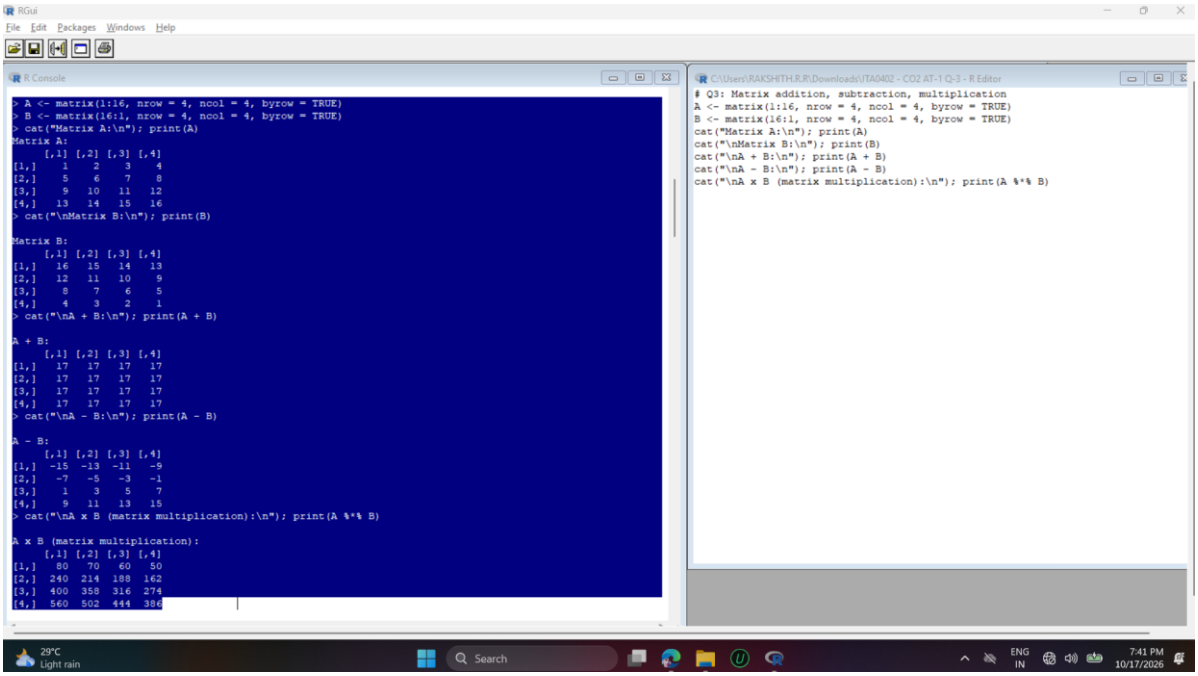
Matrix B:
     [,1] [,2] [,3] [,4]
[1,]   16   15   14   13
[2,]   12   11   10    9
[3,]    8    7    6    5
[4,]    4    3    2    1

A + B:
     [,1] [,2] [,3] [,4]
[1,]   17   17   17   17
[2,]   17   17   17   17
[3,]   17   17   17   17
[4,]   17   17   17   17

A - B:
     [,1] [,2] [,3] [,4]
[1,]  -15  -13  -11   -9
[2,]   -7   -5   -3   -1
[3,]    1    3    5    7
[4,]    9   11   13   15

A x B (matrix multiplication):
     [,1] [,2] [,3] [,4]
[1,]   80   70   60   50
[2,]  240  214  188  162
[3,]  400  358  316  274
[4,]  560  502  444  386
```

Output Screenshot:



Interpretation:

Since B is A in reverse order, every cell of A + B equals 17. A - B is element-wise. %*% performs true row-by-column matrix multiplication, which is different from the element-wise * operator.

Question 6

Write an R program using a for loop to print the Fibonacci series up to n terms, then rewrite the same using a while loop.

Approach:

The first n = 10 terms are generated with two variables a and b that hold consecutive terms. The same logic is written once with a for loop and once with a while loop.

Code:

```
# Q6: Fibonacci series using for loop and while loop
n <- 10

cat("Fibonacci using for loop:\n")
a <- 0; b <- 1
for (i in 1:n) {
  cat(a, " ")
  temp <- a + b
  a <- b
  b <- temp
}

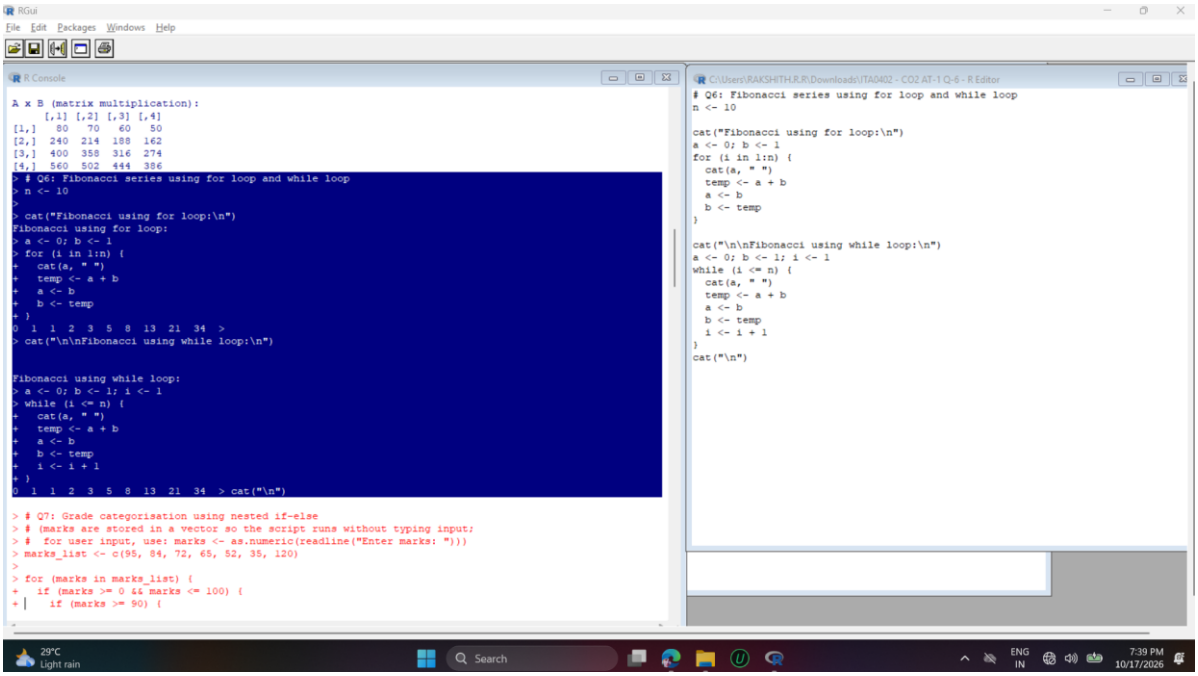
cat("\n\nFibonacci using while loop:\n")
a <- 0; b <- 1; i <- 1
while (i <= n) {
  cat(a, " ")
  temp <- a + b
  a <- b
  b <- temp
  i <- i + 1
}
cat("\n")
```

Output:

```
Fibonacci using for loop:
0  1  1  2  3  5  8  13  21  34

Fibonacci using while loop:
0  1  1  2  3  5  8  13  21  34
```

Output Screenshot:



Interpretation:

Both loops print the same series (0 1 1 2 3 5 8 13 21 34). A for loop suits a known number of terms, while a while loop repeats until a condition (i <= n) becomes false.

Question 7

Write a program using nested if-else statements to categorize a student's grade based on marks entered by the user.

Approach:

A vector of test marks is looped over. The outer if checks that the marks are valid (0 to 100); the nested if-else blocks inside it then test the grade boundaries from the highest to the lowest.

Code:

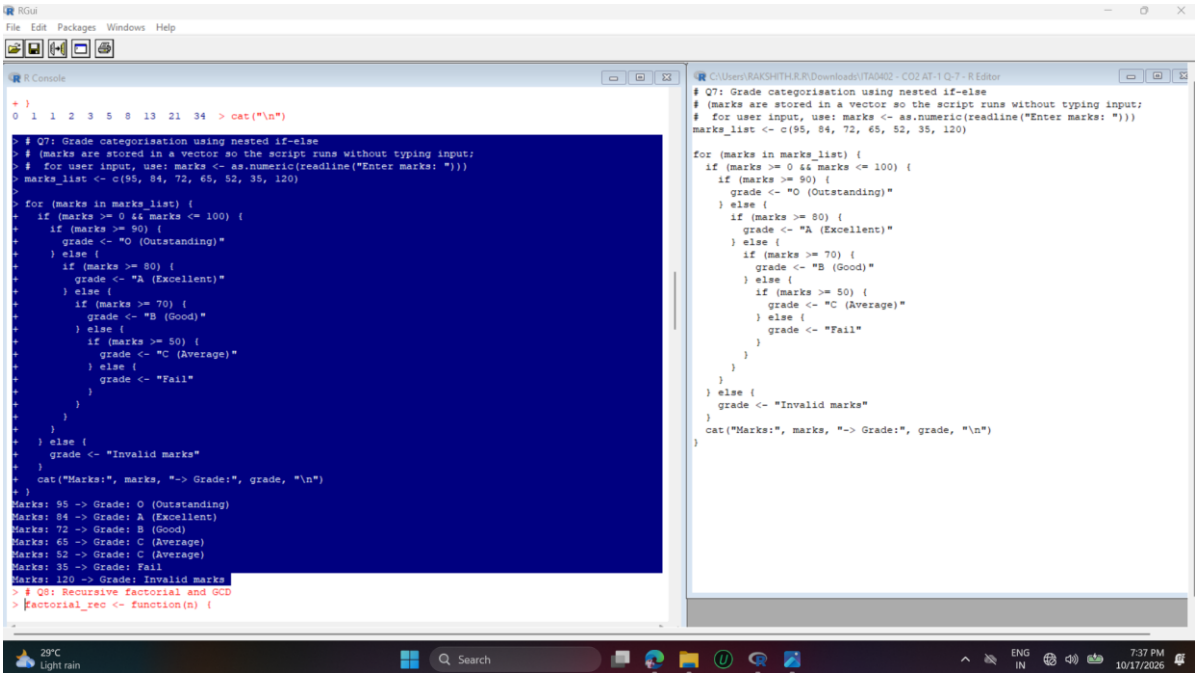
```
# Q7: Grade categorisation using nested if-else
# (marks are stored in a vector so the script runs without typing input;
#  for user input, use: marks <- as.numeric(readline("Enter marks: ")))
marks_list <- c(95, 84, 72, 65, 52, 35, 120)

for (marks in marks_list) {
  if (marks >= 0 && marks <= 100) {
    if (marks >= 90) {
      grade <- "O (Outstanding)"
    } else {
      if (marks >= 80) {
        grade <- "A (Excellent)"
      } else {
        if (marks >= 70) {
          grade <- "B (Good)"
        } else {
          if (marks >= 50) {
            grade <- "C (Average)"
          } else {
            grade <- "Fail"
          }
        }
      }
    }
  } else {
    grade <- "Invalid marks"
  }
  cat("Marks:", marks, "-> Grade:", grade, "\n")
}
```

Output:

```
Marks: 95 -> Grade: O (Outstanding)
Marks: 84 -> Grade: A (Excellent)
Marks: 72 -> Grade: B (Good)
Marks: 65 -> Grade: C (Average)
Marks: 52 -> Grade: C (Average)
Marks: 35 -> Grade: Fail
Marks: 120 -> Grade: Invalid marks
```

Output Screenshot:



Interpretation:

Marks of 95, 84 and 72 fall in grades O, A and B; 65 and 52 are grade C; 35 is Fail. The value 120 is rejected as Invalid by the outer if, which shows why the validity check is placed first.

Question 8

Write a recursive R function to calculate the factorial of a number, and another recursive function to find the GCD of two numbers.

Approach:

factorial_rec() calls itself with n - 1 until the base case (n = 0 or 1) returns 1. gcd_rec() uses Euclid's algorithm: it calls itself with (b, a %% b) until b becomes 0, and then returns a.

Code:

```
# Q8: Recursive factorial and GCD
factorial_rec <- function(n) {
  if (n < 0) return(NA)
  if (n == 0 || n == 1) return(1)
  return(n * factorial_rec(n - 1))
}

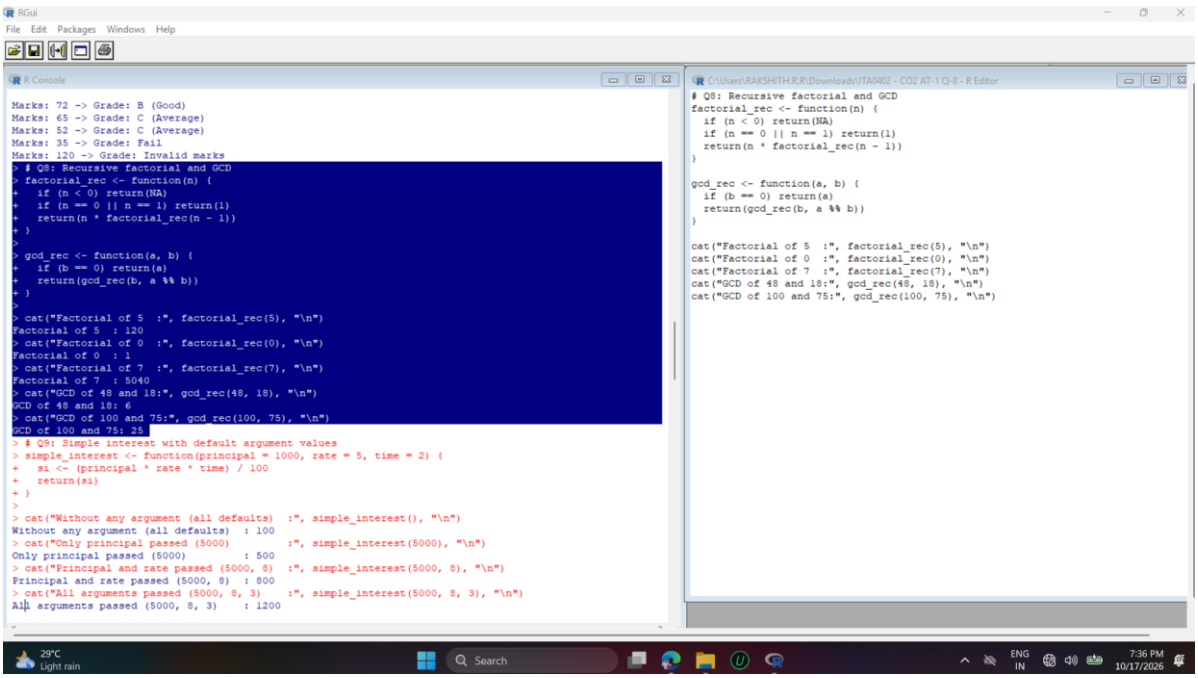
gcd_rec <- function(a, b) {
  if (b == 0) return(a)
  return(gcd_rec(b, a %% b))
}

cat("Factorial of 5  :", factorial_rec(5), "\n")
cat("Factorial of 0  :", factorial_rec(0), "\n")
cat("Factorial of 7  :", factorial_rec(7), "\n")
cat("GCD of 48 and 18:", gcd_rec(48, 18), "\n")
cat("GCD of 100 and 75:", gcd_rec(100, 75), "\n")
```

Output:

```
Factorial of 5  : 120
Factorial of 0  : 1
Factorial of 7  : 5040
GCD of 48 and 18: 6
GCD of 100 and 75: 25
```

Output Screenshot:



Interpretation:

5! = 120, 0! = 1 (base case) and 7! = 5040. GCD(48, 18) = 6 and GCD(100, 75) = 25. Each recursion stops because n decreases (factorial) or b shrinks (GCD) until the base case is reached.

Question 9

Write a function in R with default argument values that calculates the simple interest, and demonstrate calling it with and without passing all arguments.

Approach:

simple_interest() is defined with default values principal = 1000, rate = 5 and time = 2, using the formula SI = (P x R x T) / 100. It is then called with no arguments, some arguments, all arguments and named arguments.

Code:

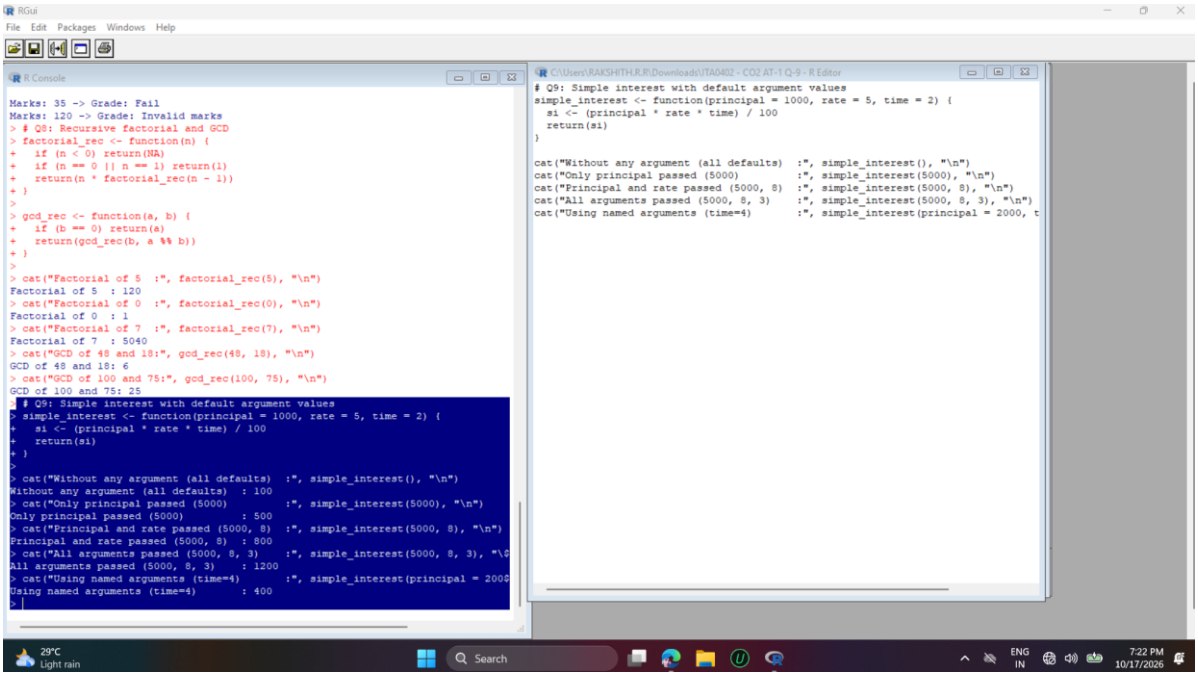
```
# Q9: Simple interest with default argument values
simple_interest <- function(principal = 1000, rate = 5, time = 2) {
  si <- (principal * rate * time) / 100
  return(si)
}

cat("Without any argument (all defaults)  :", simple_interest(), "\n")
cat("Only principal passed (5000)         :", simple_interest(5000), "\n")
cat("Principal and rate passed (5000, 8)  :", simple_interest(5000, 8), "\n")
cat("All arguments passed (5000, 8, 3)    :", simple_interest(5000, 8, 3), "\n")
cat("Using named arguments (time=4)       :", simple_interest(principal = 2000, time = 4),
"\n")
```

Output:

```
Without any argument (all defaults)  : 100
Only principal passed (5000)         : 500
Principal and rate passed (5000, 8)  : 800
All arguments passed (5000, 8, 3)    : 1200
Using named arguments (time=4)       : 400
```

Output Screenshot:



Interpretation:

When an argument is missing, R uses its default: 1000 x 5 x 2 / 100 = 100. Passing 5000 gives 500, (5000, 8) gives 800 and (5000, 8, 3) gives 1200. Named arguments (principal = 2000, time = 4) can skip rate and still give 400.

Question 10

Write a program to loop over a non-vector set (like a list of mixed data types) and print each element along with its data type.

Approach:

A list containing an integer, a decimal, a string, a logical value, a numeric vector and a complex number is looped over with seq_along(). For each element, its value, typeof() and class() are printed.

Code:

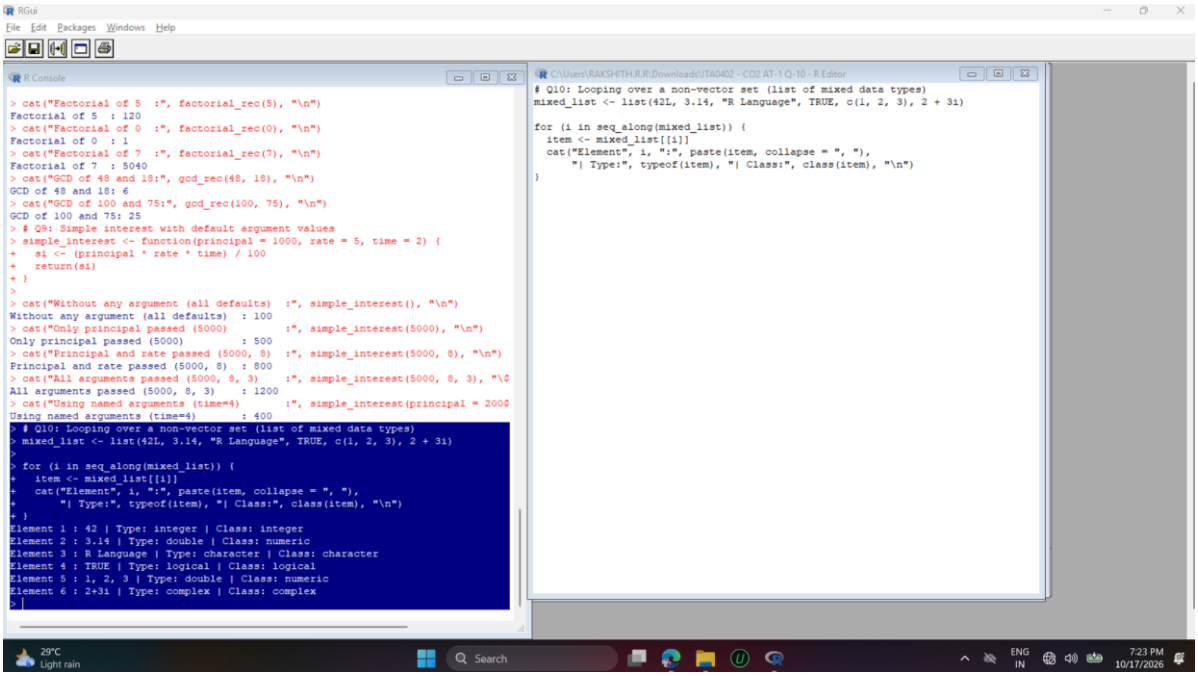
```
# Q10: Looping over a non-vector set (list of mixed data types)
mixed_list <- list(42L, 3.14, "R Language", TRUE, c(1, 2, 3), 2 + 3i)

for (i in seq_along(mixed_list)) {
  item <- mixed_list[[i]]
  cat("Element", i, ":", paste(item, collapse = ", "),
      "| Type:", typeof(item), "| Class:", class(item), "\n")
}
```

Output:

```
Element 1 : 42 | Type: integer | Class: integer
Element 2 : 3.14 | Type: double | Class: numeric
Element 3 : R Language | Type: character | Class: character
Element 4 : TRUE | Type: logical | Class: logical
Element 5 : 1, 2, 3 | Type: double | Class: numeric
Element 6 : 2+3i | Type: complex | Class: complex
```

Output Screenshot:



Interpretation:

A for loop over a list handles elements of different types one by one. typeof() shows the internal storage type (double) while class() shows the user-level class (numeric) for 3.14 and the vector c(1, 2, 3).

Question 19

Write a program to create an atomic vector of 10 numeric values and demonstrate at least 5 different vector operations (sum, mean, sorting, reversing, indexing).

Approach:

A numeric vector of 10 values is created with c(). Aggregate functions (sum, mean, max, min, length), ordering functions (sort, rev) and indexing (by position, by range and by condition) are demonstrated on it.

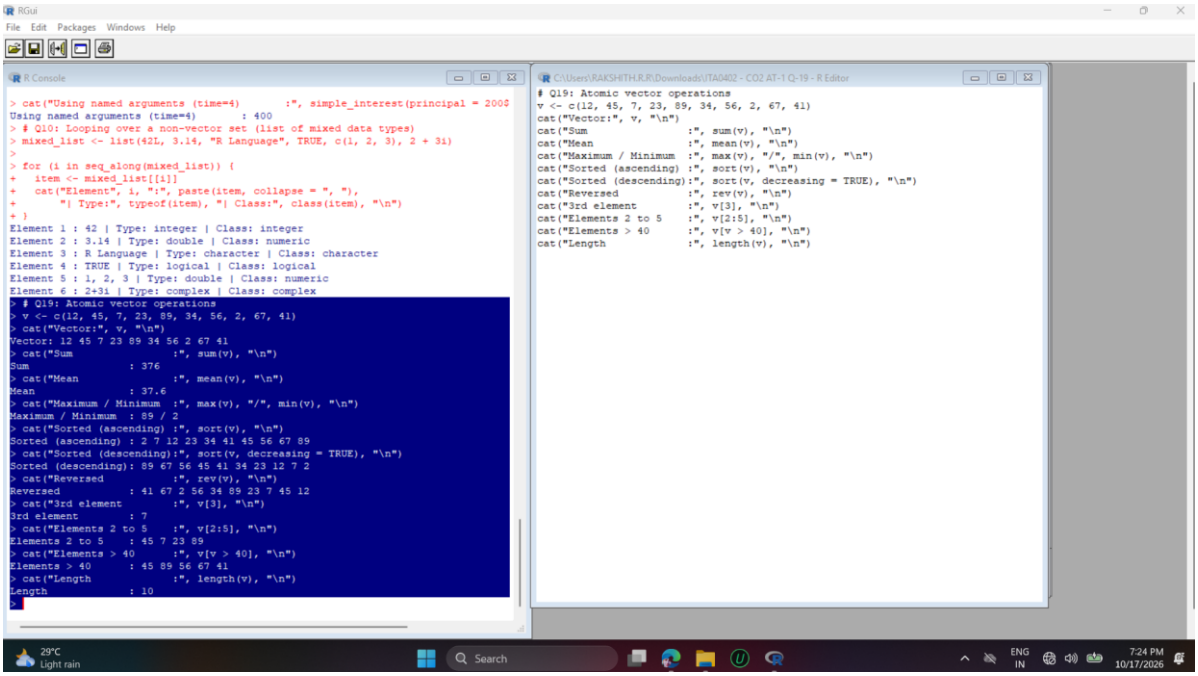
Code:

```
# Q19: Atomic vector operations
v <- c(12, 45, 7, 23, 89, 34, 56, 2, 67, 41)
cat("Vector:", v, "\n")
cat("Sum                :", sum(v), "\n")
cat("Mean               :", mean(v), "\n")
cat("Maximum / Minimum  :", max(v), "/", min(v), "\n")
cat("Sorted (ascending) :", sort(v), "\n")
cat("Sorted (descending):", sort(v, decreasing = TRUE), "\n")
cat("Reversed           :", rev(v), "\n")
cat("3rd element        :", v[3], "\n")
cat("Elements 2 to 5    :", v[2:5], "\n")
cat("Elements > 40      :", v[v > 40], "\n")
cat("Length             :", length(v), "\n")
```

Output:

```
Vector: 12 45 7 23 89 34 56 2 67 41
Sum                : 376
Mean               : 37.6
Maximum / Minimum  : 89 / 2
Sorted (ascending) : 2 7 12 23 34 41 45 56 67 89
Sorted (descending): 89 67 56 45 41 34 23 12 7 2
Reversed           : 41 67 2 56 34 89 23 7 45 12
3rd element        : 7
Elements 2 to 5    : 45 7 23 89
Elements > 40      : 45 89 56 67 41
Length             : 10
```

Output Screenshot:



Interpretation:

The sum is 376 and the mean is 37.6, with a maximum of 89 and a minimum of 2. sort() and rev() reorder the values, and logical indexing v[v > 40] correctly returns the five values greater than 40.

Question 20

Write a program to create two logical vectors and demonstrate AND, OR, NOT, and XOR operations on them element-wise.

Approach:

Two logical vectors are created and the operators & (AND), | (OR), ! (NOT) and xor() (exclusive OR) are applied element-wise. The results are collected in a data frame so each row can be compared.

Code:

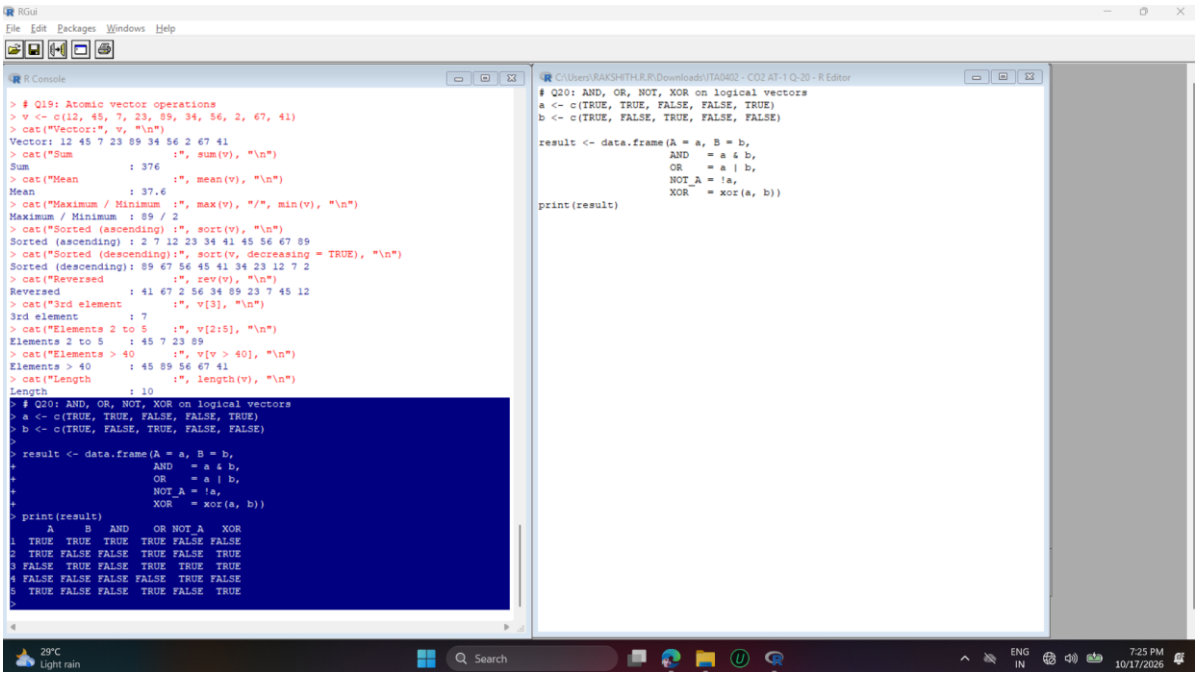
```
# Q20: AND, OR, NOT, XOR on logical vectors
a <- c(TRUE, TRUE, FALSE, FALSE, TRUE)
b <- c(TRUE, FALSE, TRUE, FALSE, FALSE)

result <- data.frame(A = a, B = b,
                     AND   = a & b,
                     OR    = a | b,
                     NOT_A = !a,
                     XOR   = xor(a, b))
print(result)
```

Output:

```
      A     B   AND    OR NOT_A   XOR
1  TRUE  TRUE  TRUE  TRUE FALSE FALSE
2  TRUE FALSE FALSE  TRUE FALSE  TRUE
3 FALSE  TRUE FALSE  TRUE  TRUE  TRUE
4 FALSE FALSE FALSE FALSE  TRUE FALSE
5  TRUE FALSE FALSE  TRUE FALSE  TRUE
```

Output Screenshot:



Interpretation:

AND is TRUE only when both values are TRUE (row 1). OR is FALSE only when both are FALSE (row 4). XOR is TRUE exactly when the two values differ, and NOT_A simply flips each value of A.

Question 21

Write a program to create a factor variable from a character vector representing blood groups, and display its levels and frequency table.

Approach:

A character vector of 12 blood groups is converted into a factor using factor(). levels() and nlevels() give the categories, and table() builds the frequency table.

Code:

```
# Q21: Factor variable, levels and frequency table
blood <- c("A+", "B+", "O+", "AB+", "O+", "A+", "B-", "O+", "A-", "AB+", "B+", "O-")
bg_factor <- factor(blood)

cat("Factor:\n"); print(bg_factor)
cat("\nLevels:", levels(bg_factor), "\n")
cat("Number of levels:", nlevels(bg_factor), "\n")
cat("\nFrequency Table:\n"); print(table(bg_factor))
```

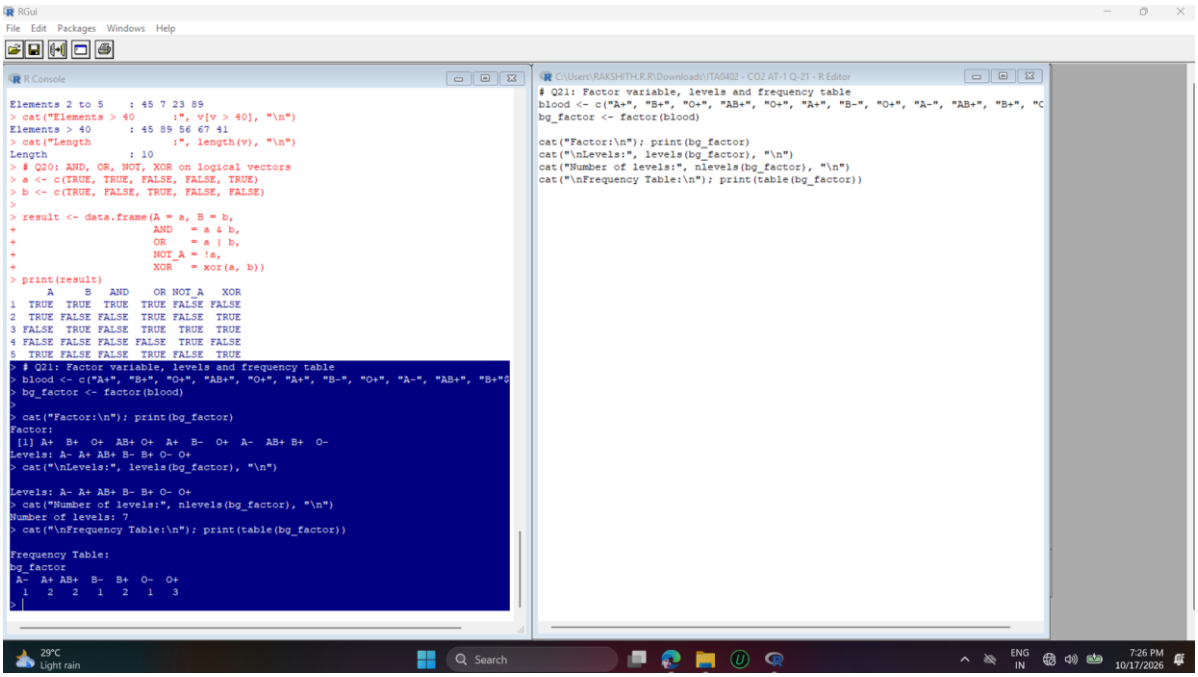
Output:

```
Factor:
 [1] A+  B+  O+  AB+ O+  A+  B-  O+  A-  AB+ B+  O-
Levels: A- A+ AB+ B- B+ O- O+

Levels: A- A+ AB+ B- B+ O- O+
Number of levels: 7

Frequency Table:
bg_factor
 A-  A+ AB+  B-  B+  O-  O+
  1   2   2   1   2   1   3
```

Output Screenshot:



Interpretation:

The 12 observations fall into 7 distinct levels, stored in alphabetical order. O+ is the most frequent blood group (3 times), while A-, B- and O- each occur once.

Question 22

Write a program to create a named list representing an employee record (name, id, salary, department) and update one field's value.

Approach:

The employee record is stored as a named list with the fields name, id, salary and department. The salary field is updated with the $ operator and the list is printed again to verify the change.

Code:

```
# Q22: Named list and updating a field
employee <- list(name = "Arun", id = 101, salary = 45000, department = "IT")
cat("Original Employee Record:\n"); print(employee)

employee$salary <- 52000
cat("\nAfter updating salary:\n"); print(employee)
cat("\nUpdated salary:", employee$salary, "\n")
```

Output:

```
Original Employee Record:
$name
[1] "Arun"

$id
[1] 101

$salary
[1] 45000

$department
[1] "IT"


After updating salary:
$name
[1] "Arun"

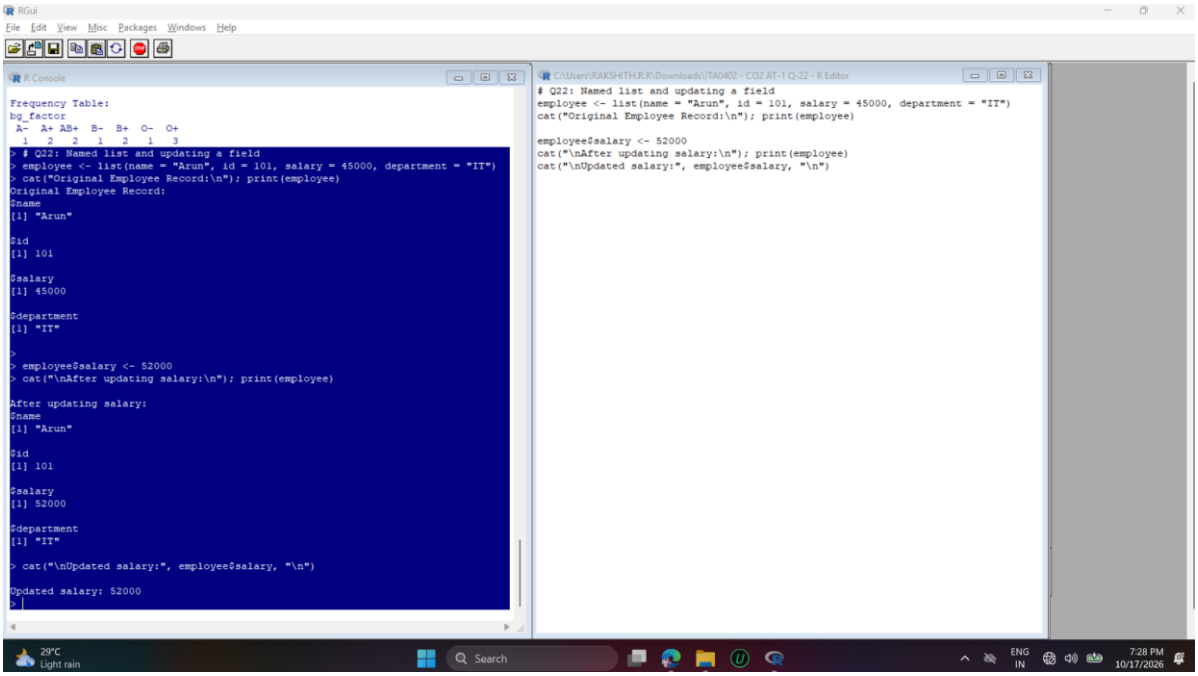
$id
[1] 101

$salary
[1] 52000

$department
[1] "IT"


Updated salary: 52000
```

Output Screenshot:



Interpretation:

Only the salary field changed (45000 to 52000); the name, id and department were unchanged. This shows that a named list element can be modified in place using employee$salary.

Question 24

Write a program to create a character vector of names, then convert it to uppercase, lowercase, and count the number of characters in each name.

Approach:

A character vector of names is converted using toupper() and tolower(), and nchar() counts the characters in each name. All results are displayed together in a data frame.

Code:

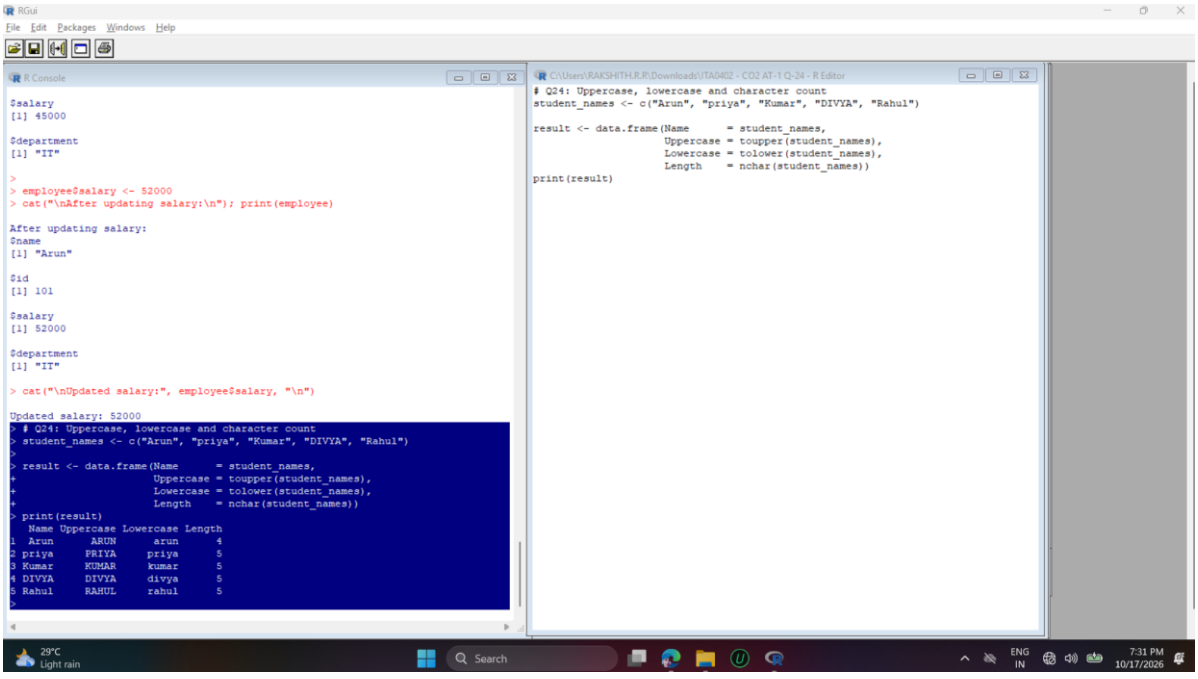
```
# Q24: Uppercase, lowercase and character count
student_names <- c("Arun", "priya", "Kumar", "DIVYA", "Rahul")

result <- data.frame(Name      = student_names,
                     Uppercase = toupper(student_names),
                     Lowercase = tolower(student_names),
                     Length    = nchar(student_names))
print(result)
```

Output:

```
   Name Uppercase Lowercase Length
1  Arun      ARUN      arun      4
2 priya     PRIYA     priya      5
3 Kumar     KUMAR     kumar      5
4 DIVYA     DIVYA     divya      5
5 Rahul     RAHUL     rahul      5
```

Output Screenshot:



Interpretation:

The string functions are vectorised, so each one processed all five names at once regardless of their original case. Arun has 4 characters and every other name has 5.

Question 27

Write a program using while loop and break/next statements to print only even numbers between 1 and 50, skipping multiples of 10.

Approach:

A while(TRUE) loop increments i each time. break ends the loop once i exceeds 50, next skips odd numbers, and a second next skips multiples of 10, so only the required even numbers are printed.

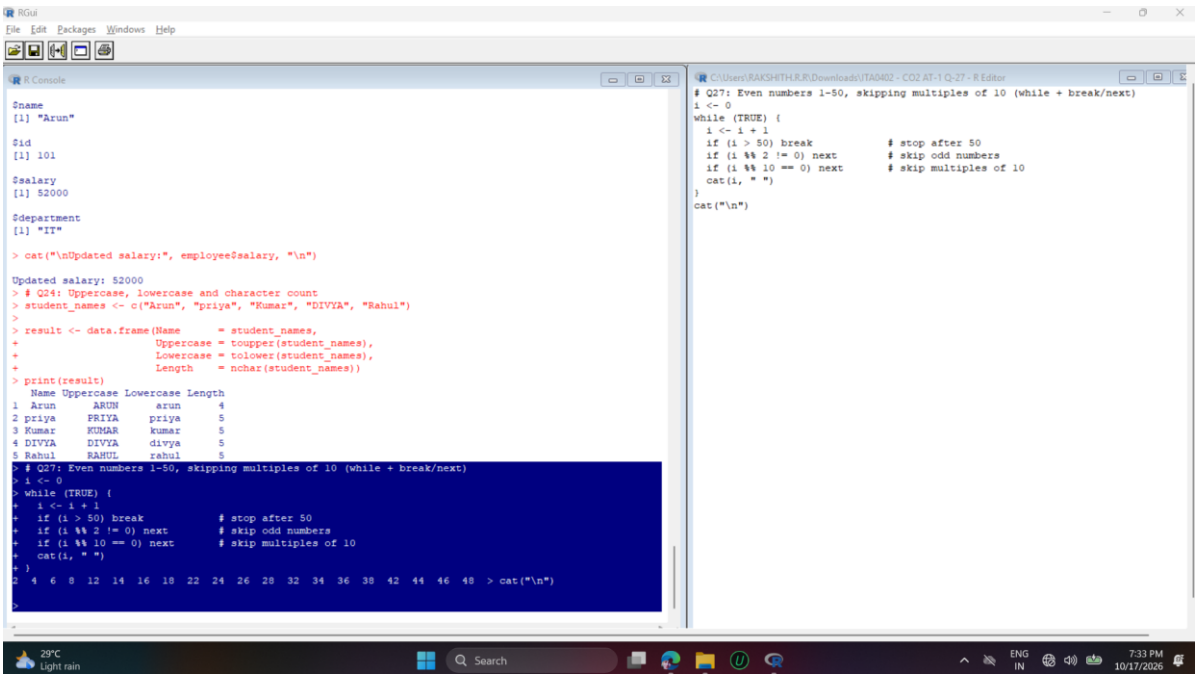
Code:

```
# Q27: Even numbers 1-50, skipping multiples of 10 (while + break/next)
i <- 0
while (TRUE) {
  i <- i + 1
  if (i > 50) break            # stop after 50
  if (i %% 2 != 0) next        # skip odd numbers
  if (i %% 10 == 0) next       # skip multiples of 10
  cat(i, " ")
}
cat("\n")
```

Output:

```
2  4  6  8  12  14  16  18  22  24  26  28  32  34  36  38  42  44  46  48
```

Output Screenshot:



Interpretation:

The output contains the even numbers from 2 to 48 and leaves out 10, 20, 30 and 40 (multiples of 10). This shows how break stops the loop and next skips the current iteration.

Question 29

Write a program with a function that returns multiple values (as a list) — minimum, maximum, and average of a numeric vector — and demonstrate calling it.

Approach:

stats_fn() packs the minimum, maximum and average of a numeric vector into a single list and returns it with return(). The caller stores the result and accesses each value by name using $.

Code:

```
# Q29: Function returning min, max and average as a list
stats_fn <- function(x) {
  return(list(minimum = min(x), maximum = max(x), average = mean(x)))
}

data <- c(23, 45, 12, 67, 34, 89, 56)
res <- stats_fn(data)
cat("Data:", data, "\n")
print(res)
cat("Minimum:", res$minimum, "\n")
cat("Maximum:", res$maximum, "\n")
cat("Average:", res$average, "\n")
```

Output:

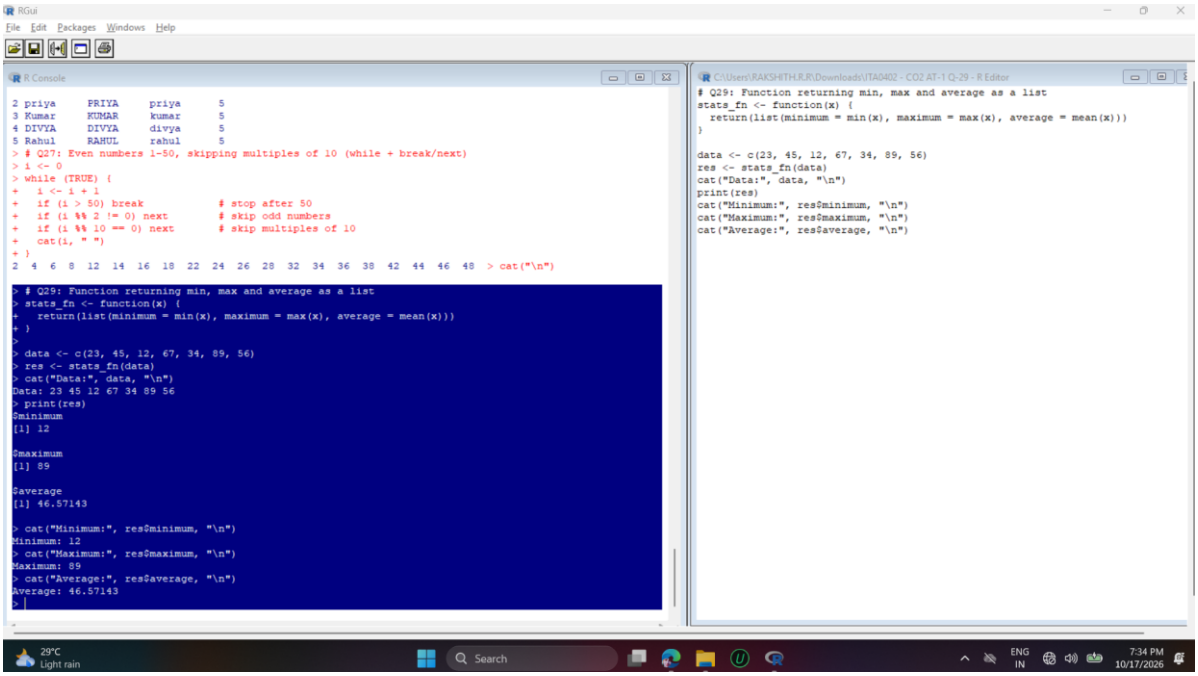
```
Data: 23 45 12 67 34 89 56
$minimum
[1] 12

$maximum
[1] 89

$average
[1] 46.57143

Minimum: 12
Maximum: 89
Average: 46.57143
```

Output Screenshot:



Interpretation:

For the data 23, 45, 12, 67, 34, 89, 56 the function returned a minimum of 12, a maximum of 89 and an average of 46.57 (326 / 7). A list lets one function return several values of different kinds.